

Sistemas Operativos

Chamadas de Sistema

Interrupções

Processos

Tarefas

1

1

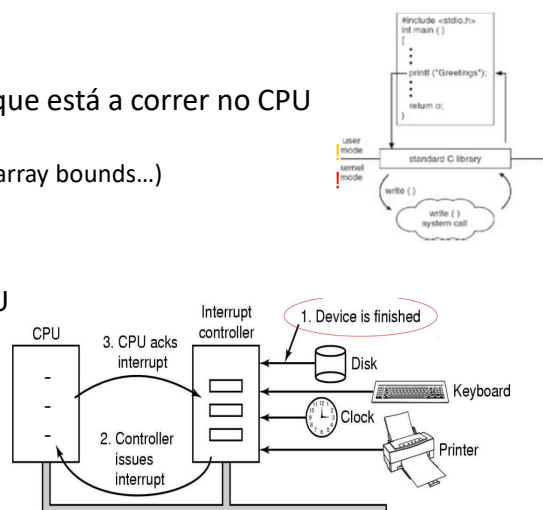
Razões para passar de modo utilizador para modo núcleo?

- Traps

- Interrupção de **SW** – pelo processo que está a correr no CPU
 - Chamadas de sistema – **System Calls**
 - Exceções (por aplicação: division by 0; array bounds...)

- Interrupções

- Interrupção de **HW** – externo ao CPU
 - Disco
 - ...E/S

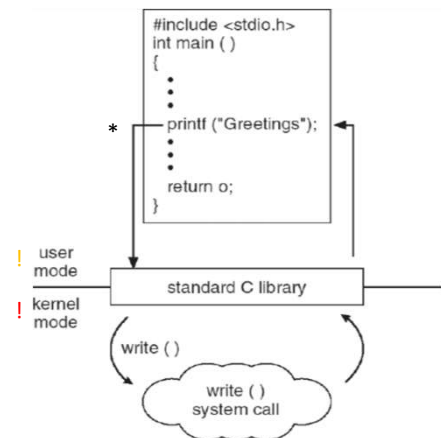


2

2

System Calls

- São solicitações ao SO pelo programa que está a correr *
- Causam uma TRAP – neste caso interrupção por system call (...próximos slides...)
- Passa do modo utilizador para o modo núcleo
- Recebe pedido, processa e retorna para continuar programa após origem da chamada

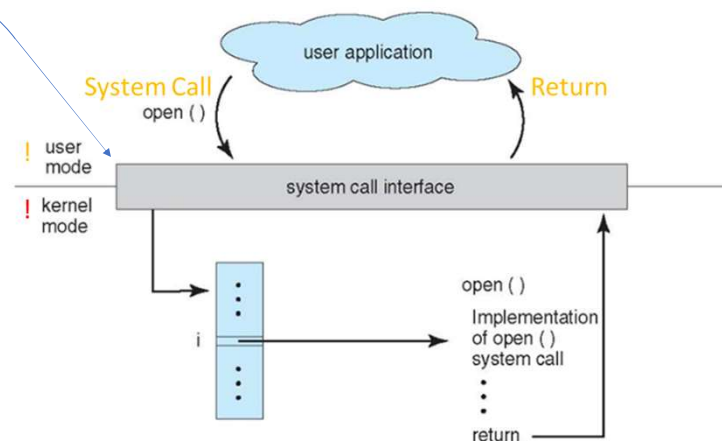


3

3

System Calls -Interface

- A interface encapsula o acesso direto às chamadas de sistema.
- Esconde a complexidade
- Disponibilizada pelo SO



4

4

System Calls - Exemplos

- System Calls – Chamadas de Sistema

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

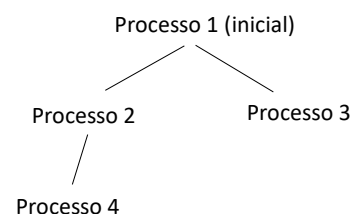
5

5

Processos

- Cada instância de um programa em execução denomina-se por processo
- Organizam-se normalmente de forma hierárquica:
 - 1 processo ao criar outro, cria uma ligação pai-filho (**partilha e herança de propriedades**)*

Estrutura hierárquica dos processos



6

6

Processos em Execução – Windows

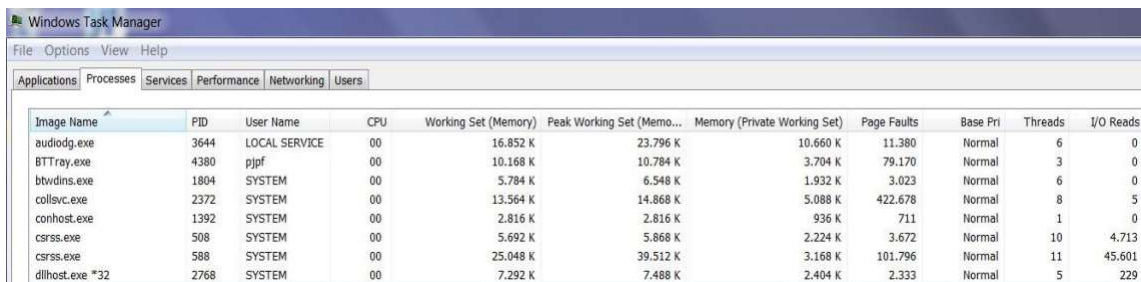


Image Name	PID	User Name	CPU	Working Set (Memory)	Peak Working Set (Memo...	Memory (Private Working Set)	Page Faults	Base Pri	Threads	I/O Reads
audiodg.exe	3644	LOCAL SERVICE	00	16.852 K	23.796 K	10.660 K	11.380	Normal	6	0
BTTray.exe	4380	pjpf	00	10.168 K	10.784 K	3.704 K	79.170	Normal	3	0
btwdins.exe	1804	SYSTEM	00	5.784 K	6.548 K	1.932 K	3.023	Normal	6	0
collsvc.exe	2372	SYSTEM	00	13.564 K	14.868 K	5.088 K	422.678	Normal	8	5
conhost.exe	1392	SYSTEM	00	2.816 K	2.816 K	936 K	711	Normal	1	0
csrss.exe	508	SYSTEM	00	5.692 K	5.868 K	2.224 K	3.672	Normal	10	4,713
csrss.exe	588	SYSTEM	00	25.048 K	39.512 K	3.168 K	101.796	Normal	11	45,601
dllhost.exe *32	2768	SYSTEM	00	7.292 K	7.488 K	2.404 K	2.333	Normal	5	229

7

7

Processos em Execução – Unix

```
ps -el | more
  UID    PID  PPID  C   STIME TTY   TIME CMD
  root      0      0  0   Sep 18 ?    0:17 sched
  root      1      0  0   Sep 18 ?    0:54 /etc/init -
  root      2      0  0   Sep 18 ?    0:00 pageout
  root      3      0  0   Sep 18 ?    6:15 fsflush
  root    418      1  0   Sep 18 ?    0:00 /usr/lib/saf/sac -t 300
daemon  156      1  0   Sep 18 ?    0:00 /usr/lib/nfs/statd
```

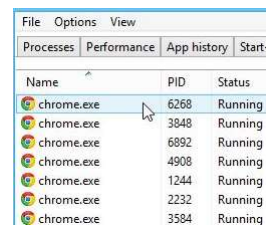
```
ps      displays information about a selection of the
active processes.
e       select all processes
l       long format
```

8

8

Programa vs. Processo (?)

- Programa = ficheiro executável (passivo)
- Processo = imagem viva de um programa em execução (ativo)
 - Inclui informação sobre:
 - PID
 - PPID
 - Memória alocada
 - Estado
 - ...
 - Contexto HW e SW

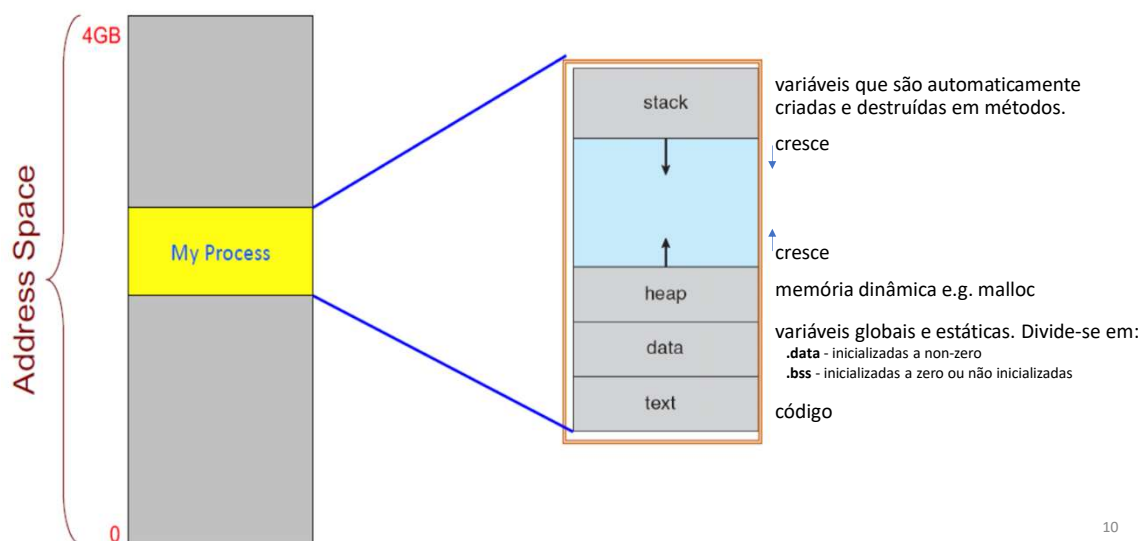


Name	PID	Status
chrome.exe	6268	Running
chrome.exe	3848	Running
chrome.exe	6892	Running
chrome.exe	4908	Running
chrome.exe	1244	Running
chrome.exe	2232	Running
chrome.exe	3584	Running

9

9

Memória de um Processo

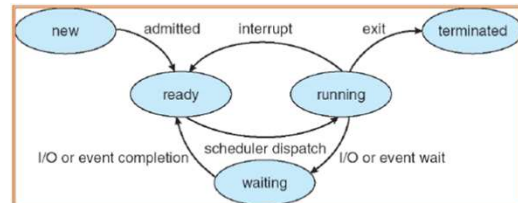
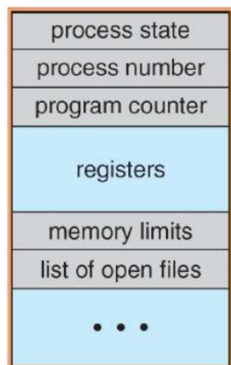


10

10

Bloco de Controlo de Processo - PCB process control block

- Detalha um processo no SO



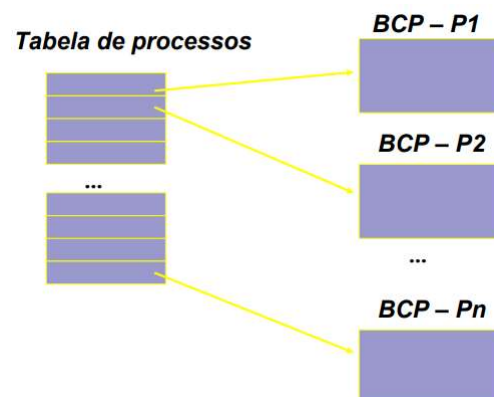
- Vital para a gestão de processos

11

11

Bloco de Controlo de Processo - PCB process control block

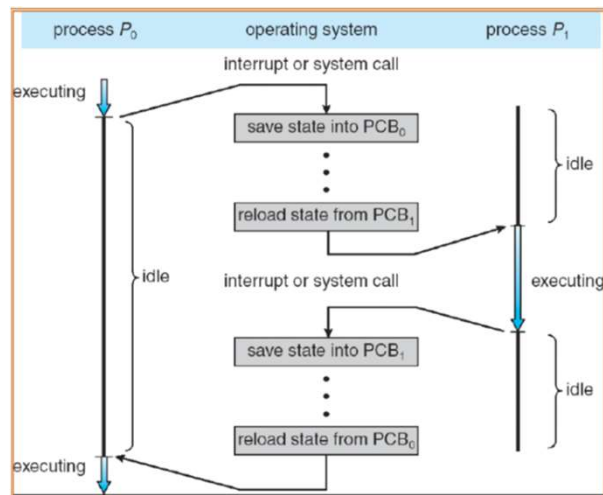
- Existem inúmeros processos
- Cada processo tem uma entrada na tabela de processos
- Essa entrada aponta para os detalhes desse processo



12

12

Processos em 'Paralelo'

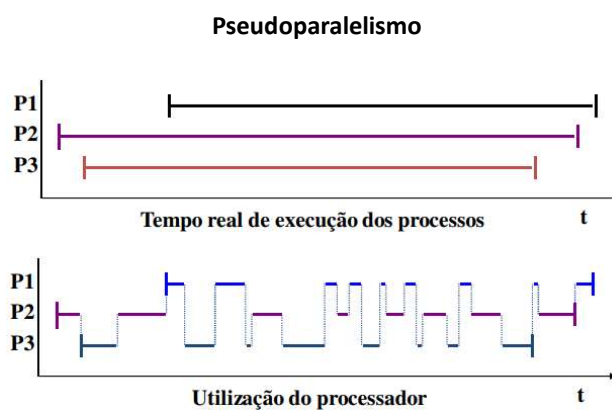


13

13

Processos em 'Paralelo'

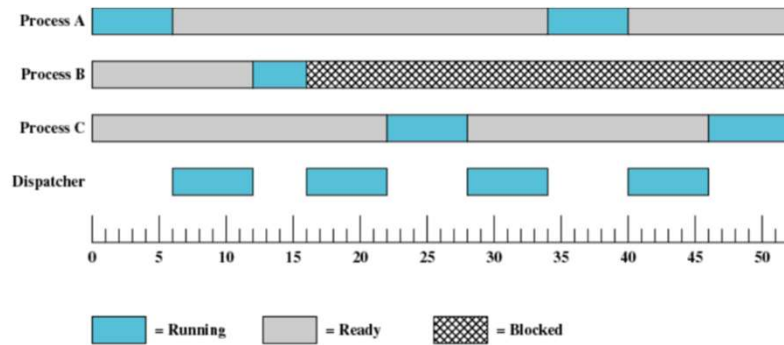
- Em multiprogramação são executados em 'paralelo' (pseudoparalelismo*)



14

14

Processos em 'Paralelo'



- Apenas os processos no estado **Running** ocupam o CPU!
(próximas aulas: scheduler e dispatcher, o que fazem?)

15

15

Tipos de Processos

- I/O Bound
 - Utiliza mais I/O (E/S) do que CPU
- CPU Bound
 - Utiliza mais CPU do que E/S

Deve haver um bom equilíbrio entre a execução dos 2 tipos de processos – porquê?

16

16

Criação de Processos

- Os processos estão constantemente a ser criados:
 - Startup do sistema
 - Pedido do utilizador (e.g. 'abre' aplicação)
 - ...
 - Processos em 2º plano (BG)
 - Serviços de impressão
 - Serviços de email
 - ...
- Vamos também lançar novos processos em C. Como?

17

17

Criação de Processos - Unix

Cada processo tem um identificador (PID – Process ID). Cada processo tem um pai que também tem um identificador (PPID)

- `pid_t getpid(void);`
 - Retorna o **PID** do processo atual
- `pid_t getppid(void);`
 - Retorna o PID do pai do processo atual (**PPID**)
- `pid_t fork(void);`
 - Cria novo processo. Ao processo pai é retornado o PID do filho e ao filho é retornado 0 (retorna 2 valores?)
 - Retorna -1 em caso de erro.
- `pid_t wait(int* status);` tipicamente `-> wait(NULL)`
 - Espera até um filho terminar (retorna o PID do filho). Estado do filho é recebido em `status` (signal).
 - Se não houver filhos, ignora o `wait`
- `pid_t waitpid(pid_t who, int* status, int options);`
 - Semelhante a `wait()`, mas permite esperar por um filho específico. A flag `WNOHANG` permite verificar se o referido processo já terminou e saltar o bloqueio.

18

18

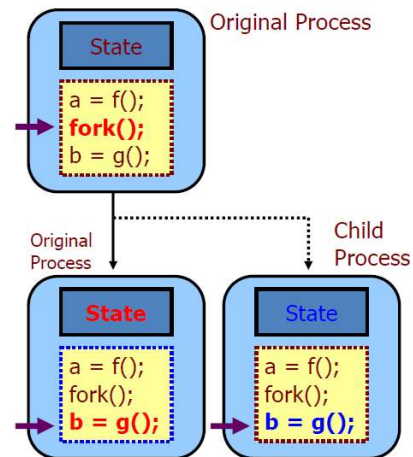
Criação de Processos - Unix

[System call (trap)]

Comando para criar um processo:

- **fork()**

- Retorna o PID `pid=fork()`
- Cria um **processo filho que fica com uma cópia do contexto do pai**
- Alterar uma variável no filho não altera no pai (e vice-versa)
- Os Prog. Counters passam a apontar para a instrução seguinte.



19

19

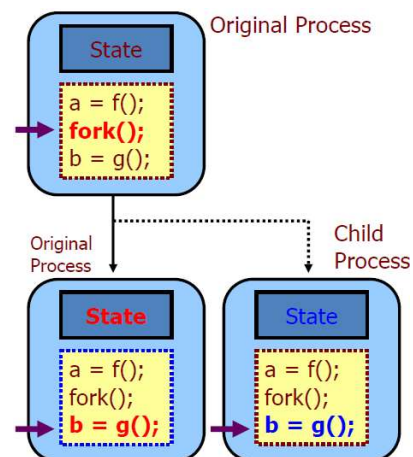
Criação de Processos - Unix - Exemplo

```
int main()
{
    printf("Start of the main function...\n");

    int id, i;
    id=fork();
    if(id==0)
    {
        /*child process*/
        printf("\nChild section...\n");
        i=1;
    }
    else if(id>0)
    {
        /*parent process*/
        printf("\nParent section...\n");
        i=0;
        //wait(NULL);
    }

    printf("End of the main function...%d\n", i);

    return 0;
}
```



20

20

Criação de Processos - Unix - Exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    fork();
    fork();

    printf("[%d] is my PID\n", getpid());

    return 0;
}
```

Qual o output?

21

21

Terminação de Processos

Pode terminar quando:

- A tarefa a ser executada é finalizada (dito normal)
- Invocando `exit` (UNIX) e `ExitProcess` (Windows)
- Ocorre um erro (e.g. divisão por 0)

Depois de terminar:

- Liberta todos os recursos detidos pelo processo
- Assinala a terminação ao processo pai

22

22

Terminação de Processos

```
main() {
    int pid, estado;
    pid = fork ();

    if (pid == 0) {
        /* processo filho */
        exit(0);
    }
    else {
        /* pai bloqueia à espera
        que o filho termine*/
        pid = wait(&estado);
    }
}
```

- `exit` causa eliminação de grande parte do estado do processo
- Só depois de `wait` é que o processo é totalmente eliminado
- i.e. faz `exit`, mas são mantidos atributos que serão necessários para quando o pai chamar `wait`:
 - PID do processo terminado e do seu processo pai
 - Status da terminação
- ∴ Entre `exit` e `wait`, o processo diz-se zombie

23

23

Terminação de Processos

Um processo só é mesmo terminado pelo SO quando o pai chama `wait/waitpid` (permite ao pai verificar código de saída)

- **Processo Zombie**
 - É um processo que 'morre' (via `exit`) mas não é reconhecido pelo pai (ausência de `wait`).
 - Pode ficar a consumir recursos.
- **Processo Órfão**
 - É um processo cujo pai 'morre'.
 - É automaticamente 'adotado' por um processo de sistema.

24

24

Exemplo de Processo Zombie ou Órfão?

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

void worker()
{
    printf("[%d] Hi, I'm a worker child process! Going to die...\n", getpid());
}

int main()
{
    for (int i=0; i<3; i++)
    {
        if (fork() == 0)
        {
            worker();
            exit(0);
        }

        printf("[%d] Big father is sleeping!\n", getpid());
        sleep(10);
    }

    return 0;
}
```

Nota: no final devíamos fazer waits. Código apenas para exemplificar zombie/órfão.

Qual o output?

Comando para listar processos:
`ps -el`

25

25

Exemplo de Processo Zombie ou Órfão?

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

void worker()
{
    sleep(10);
    printf("[%d] Let's see who my dady is: %d\n", getpid(), getppid());
}

int main()
{
    for (int i=0; i<3; i++)
    {
        if (fork() == 0)
        {
            worker();
            exit(0);
        }

        printf("[%d] Big father is going away!\n", getpid());
    }

    return 0;
}
```

Nota: no final devíamos fazer waits. Código apenas para exemplificar zombie/órfão.

Comando para listar processos:
`ps -el`

26

26

Processo invoca outro Processo

- É possível substituir a imagem do atual processo por outra, chamando a ‘família’ de funções `exec()`
 - É tudo substituído pelo novo executável (código, stack, heap...)!
 - `int execl(const char *path, const char *arg, ...);`
 - `int execlp(const char *file, const char *arg, ...);`
 - `int execl(const char *path, const char *arg, ..., char *const envp[]);`
 - `int execv(const char *path, char *const argv[]);`
 - `int execvp(const char *file, char *const argv[]);`
- Funções com ‘p’: usam a PATH do ambiente
- Funções com ‘v’: usam um apontador para um array de parâmetros
- Funções com ‘l’: parâmetros separados por vírgulas
- O primeiro parâmetro é o nome do programa.

27

27

Processo invoca outro Processo - Exemplo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
```

```
int main()
{
    if (execlp("ls", "ls", "-a", NULL) == -1) {
        perror("Error executing ls: ");
    }
    else {
        printf("This cannot happen!\n");
    }
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
```

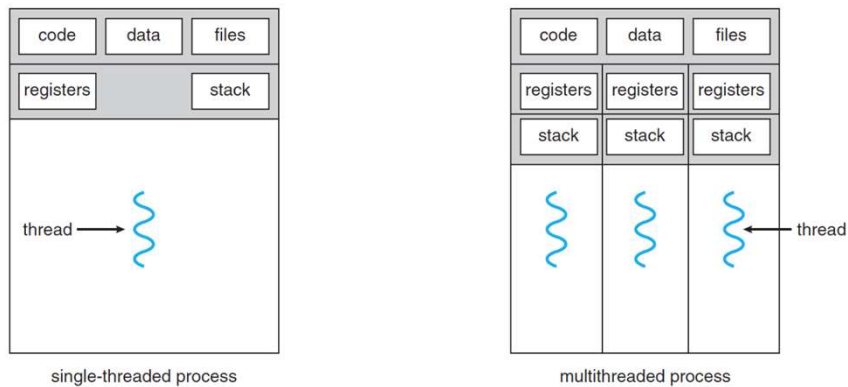
```
int main()
{
    char* ls_param[] = { "ls", "-a", NULL };
    if (execvp(ls_param[0], ls_param) == -1) {
        perror("Error executing ls: ");
    }
    else {
        printf("This cannot happen!\n");
    }
    return 0;
}
```

28

28

Tarefas

- threads = fios (de execução) independentes que partilham um contexto comum (de um processo).

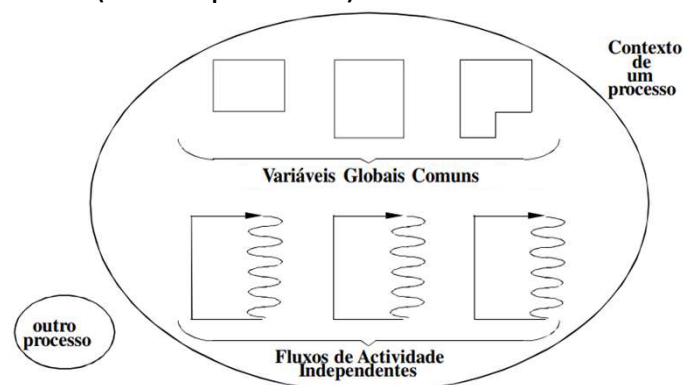


29

29

Tarefas

- threads = fios (de execução) independentes que partilham um contexto comum (de um processo).



30

30

Tarefas

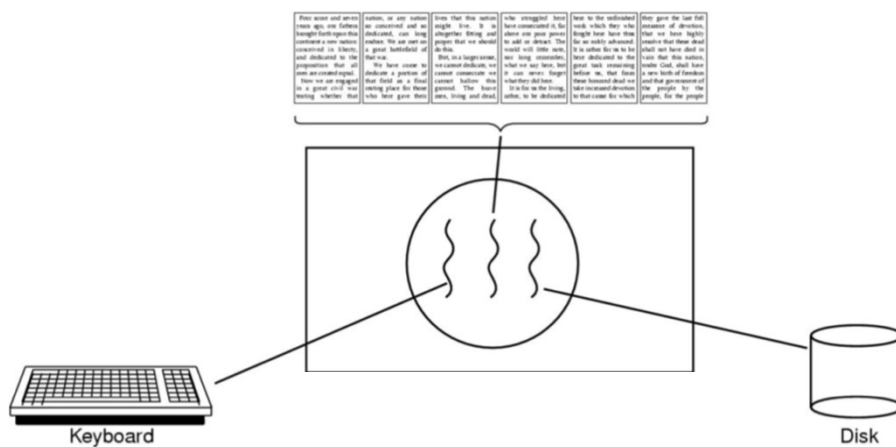
Porque não usar processos? Qual a motivação para usar threads?

- São fluxos de execução dentro de um mesmo processo
- O espaço de memória é o mesmo (partilha espaço de memória)
 - **Não há mudança de contexto** como há quando comuta processos
 - A comutação e comunicação entre tarefas é mais eficiente
 - **NOTA: a alteração de uma variável global dentro de uma tarefa reflete-se em todas as outras. As alterações ficam logo disponíveis para todas as tarefas.**
- ...são mais 'leves' que os processos
- ...é preciso **sincronizar acessos!!!!**

31

31

Tarefas

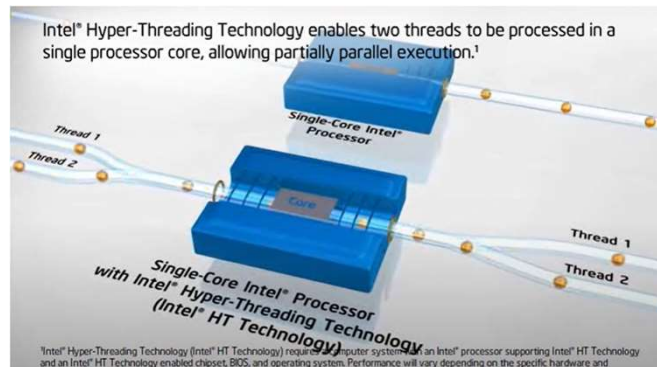


32

32

Multi-CPU vs Multi-Core vs Hyper-Threading

- Multi Processador
 - $\neq$ CPU's
 - $N \text{ CPU} = N \text{ tarefas em paralelo}$
- Multi-Core
 - e.g. dual core – 2 tarefas
 - 1 CPU
 - 2 cores
 - 2 tarefas em paralelos
- Hyper-Threading
 - e.g. dual core – 4 tarefas
 - 1 CPU (2 cores)
 - 1º core
 - 2 tarefas (pseudo paralelo)
 - 2º core
 - 2 tarefas (pseudo paralelo)



33

33

Tipos de Tarefas

- No **modo utilizador** (user threads)
 - As tarefas são geridas pelo utilizador, sem suporte do núcleo
 - Mais rápidas de criar e gerir
- No **modo núcleo** (kernel threads)
 - Geridas diretamente pelo do sistema operativo
 - Mais lentas de criar e gerir

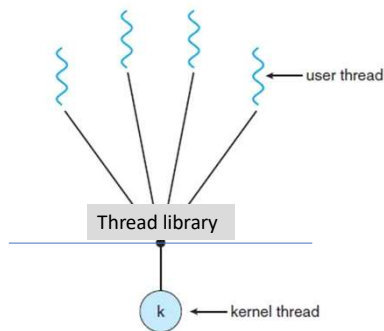
É necessário modelar a relação entre os dois tipos de tarefas...

...Modelos de Multithreading (N para 1; 1 para 1; N para N)

34

34

Modelos de Multithreading



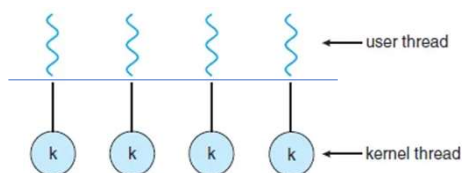
N para 1

- N tarefas no MU são mapeadas para 1 no MN
- A gestão é feita através de 1 biblioteca no MU (eficiente)
- Quando 1 tarefa faz uma *system call* que bloqueia, todo o processo bloqueia
- Apenas uma de cada vez consegue aceder ao MN (paralelismo?)

35

35

Modelos de Multithreading



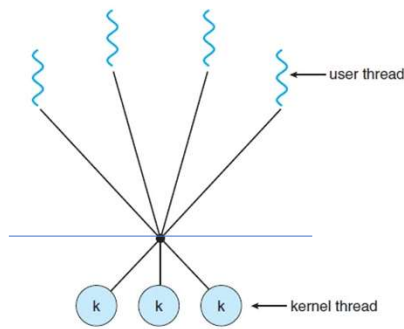
1 para 1

- Cada tarefa no MU é mapeada para 1 tarefa no MN
- Quando 1 tarefa faz uma *system call* que bloqueia, as outras não são afetadas
- Permite que as tarefas sejam executadas em paralelo
- Criar 1 tarefa no MU implica criar também 1 no MN (performance?)
- Poderá haver um número limite de tarefas que é possível criar

36

36

Modelos de Multithreading



N para N

- N tarefas no MU são mapeadas para um número = ou < de tarefas no MN
- Programador pode criar no MU quantas tarefas necessitar e as tarefas MN correspondentes podem correr em paralelo
- Quando 1 tarefa faz uma *system call* que bloqueia, o núcleo pode escalonar outra tarefa para execução
- O número de tarefas no MN pode ser específico para determinada aplicação ou máquina

37

37

Criação de Threads

- `pthread_create(&tid, attr, function, arg)`
 - Apontador para o identificador da tarefa, atributos, função a executar, arg
- `pthread_exit(void *value_ptr)`
- `int pthread_join(pthread_t t, void **value_ptr)`
 - suspende a tarefa invocadora até "pthread_t" terminar; continua a execução caso "pthread_t" já tenha terminado (i.e. espera que determinada tarefa termine).
- **Compilar com a opção -pthread:**
 - `gcc -g -Wall -pthread nome.c -o nomeprog`

38

38

Criação de Threads – Exemplo

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *slow_thread() {
    for(int i=0; i<3; i++)
    {
        printf("Hello, I'm Sllloooooooowww thread \n");
        sleep(3);
    }

    pthread_exit(NULL);
    return NULL;
}

void *fast_thread() {
    for(int i=0; i<3; i++)
    {
        printf("Hello, I'm FAST thread \n");
        sleep(1);
    }

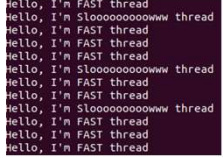
    pthread_exit(NULL);
    return NULL;
}

int main(void) {
    pthread_t thr_slow, thr_fast;

    pthread_create(&thr_slow, NULL, slow_thread, NULL);
    pthread_create(&thr_fast, NULL, fast_thread, NULL);

    pthread_join(thr_slow, NULL);
    pthread_join(thr_fast, NULL);

    return 0;
}
```



39

39

Criação de Threads – Exemplo

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 10

pthread_t my_thread[N];
int id[N];

void *worker(void* idp) {
    int my_id = *((int *)idp);

    printf("Hello, I'm thread %d\n", my_id);
    sleep(rand() % 3);
    printf("Hello, I'm thread %d, going away!\n", my_id);

    pthread_exit(NULL);
    return NULL;
}

int main(void) {
    int i;
    // create N threads
    for (i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&my_thread[i], NULL, worker, &id[i]);
    }

    //waits for each of them to die
    for (i = 0; i < N; i++) {
        pthread_join(my_thread[i], NULL);
    }

    return 0;
}
```



40

40

Criação de Threads – Exemplo execução sequencial*

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define TOTAL 1000000000
#define TOTALBY4 TOTAL/4

unsigned long sum[4] = {};
int id[4] = {};

void addAll(int index){
    int i = index;
    //sum count 0-25
    sum[i]=0;

    while(sum[i] < TOTALBY4)
    {
        sum[i] ++;
    }

    printf("run -> %d\n",i);
}

int main(void){
    int i=0;
    for(i=0; i<4; i++)
    {
        id[i] = i;
        addAll(id[i]);
    }

    printf("sum = %llu\n", sum[0] + sum[1] + sum[2] + sum[3]);

    return 0;
}
```

time ./nomeprog

```
run -> 0
run -> 1
run -> 2
run -> 3
sum = 1000000000
real    0m1,900s
```

41

41

Criação de Threads – Exemplo execução em paralelo*

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define TOTAL 1000000000
#define TOTALBY4 TOTAL/4

unsigned long sum[4] = {};
int id[4] = {};

void *thread_addAll(void *index){
    int i = *((int *)index);
    //sum count 0-25
    sum[i]=0;

    while(sum[i] < TOTALBY4)
    {
        sum[i] ++;
    }

    printf("run -> %d\n",i);

    return NULL;
}

int main(void){
    pthread_t my_threads[4];
    int i=0;
    for(i=0; i<4; i++)
    {
        id[i] = i;
        pthread_create(&my_threads[i], NULL, thread_addAll, &id[i]);
    }

    for(i=0; i<4; i++)
    {
        pthread_join(my_threads[i], NULL);
    }

    printf("sum = %llu\n", sum[0] + sum[1] + sum[2] + sum[3]);

    return 0;
}
```

```
run -> 3
run -> 1
run -> 2
run -> 0
sum = 1000000000
real    0m0,891s
```

42

42

Terminação de Threads

- `pthread_exit(NULL);`
- Quando a `main()` termina, todas as tarefas em execução terminam...
...se for necessário terminar apenas a thread `main()`:

```
int main(void){  
    pthread_t tid_teste;  
    pthread_create(&tid_teste, NULL, thread_teste, NULL);  
    pthread_exit(NULL);  
    printf("Main thread killed. Can you see me????");  
    return 0;  
}
```

→ `pthread_exit(NULL);` → ∴ A `main()` faz `exit`, mas as tarefas 'abertas' continuam a sua execução